

SISTEMAS COMPUTACIONAIS

Pipeline

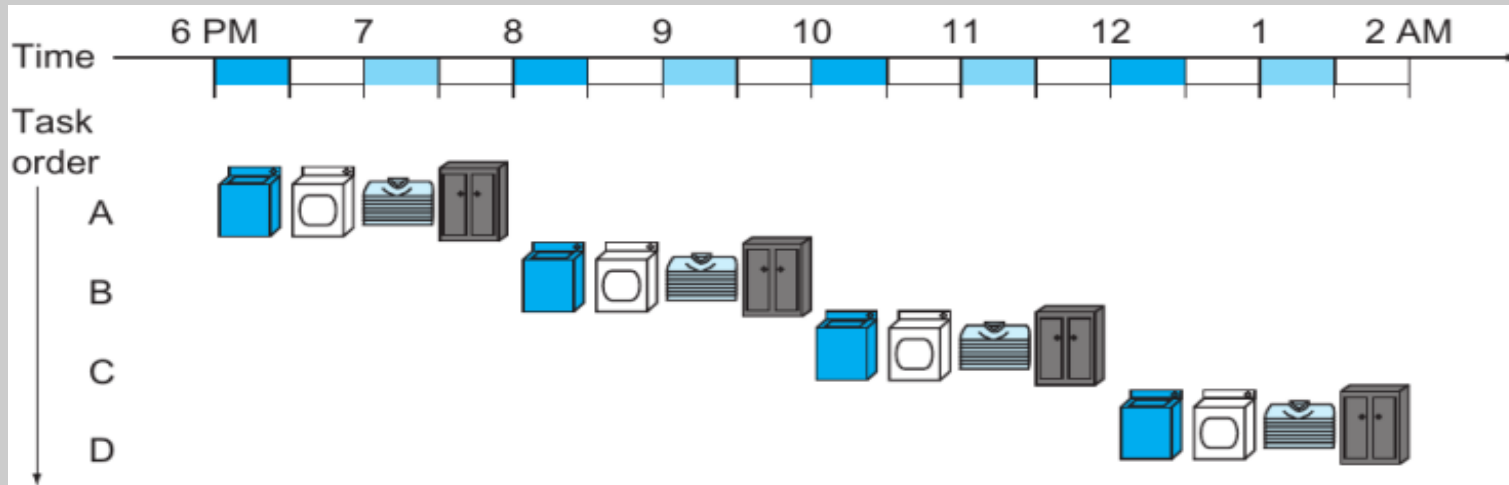
BUSCA DE DESEMPENHO

- Objetivo: melhorias de desempenho
 - ✓ melhorias na tecnologia (ex. circuitos mais rápidos)
 - ✓ melhorias organizacionais no processador com múltiplos registradores e memória cache (L1 e em níveis)

Pipeline (de instrução): várias instruções são realizadas de forma simultânea, embora em estágios diferentes, pois múltiplas instruções são emitidas por ciclo de *clock*

- A CPU se divide em várias partes funcionais distintas (estágios), cada uma correspondendo a uma determinada ação
- A ideia é de que novas entradas são aceitas, antes que as entradas aceitas previamente tenham terminado, o que assume uma instrução de vários estágios

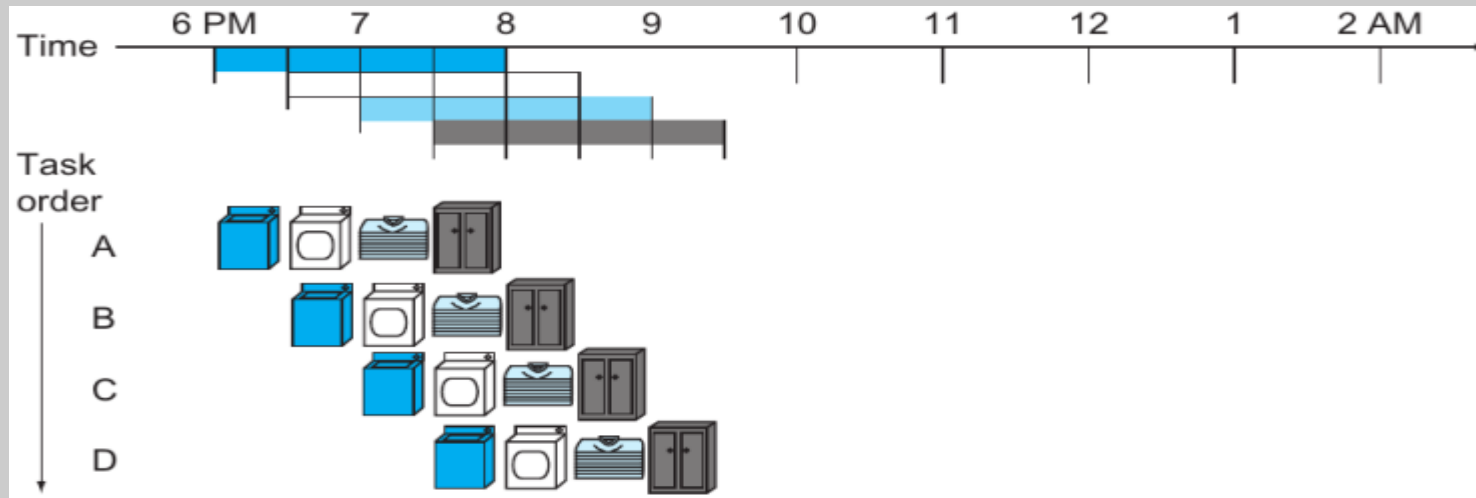
LAVAR ROUPA SEM PIPELINE



Na lavanderia (CPU), a tarefa (a instrução) de lavar roupa tem 4 estágios (ações):

1. Colocar uma carga de roupa na lavadora. (L) (1t)
 2. Quando terminar de lavar, colocar na secadora. (S)(1t)
 3. Quando terminar de secar, dobrá-las. (D)(1t)
 4. Quando terminar de dobrar, armazenar no cesto (A) (1t)
- 4t depois, começar a lavar de novo,
 - Para 4 cestos (A,B,C,D), temos $4 \times 4t = 16t$ (sem novos recursos)

LAVAR ROUPA COM PIPELINE



Após a máquina de lavar terminar o cesto A, transfere-se o cesto para a secadora

- O próximo cesto já é inserido na máquina de lavar e, enquanto a secadora “processa”, continua o processo com a trouxa anterior
- A máquina de lavar nunca fica ociosa
- Faz-se o mesmo para os outros
- Para 4 cestos (A,B,C,D), temos $7t$ (sem novos recursos e t igual)

PIPELINE

- Considere um pipeline com 6 estágios de mesma duração

Início

busca de instrução (FI – *Fetch Instruction*)

decodificação de instrução (DI)

cálculo de operandos (CO)

busca de operandos (FO – *Fetch Operands*)

execução de instrução (EI)

escrita de operando (WO – *Write Operand*)

Fim

PIPELINE

	Tempo →													
	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instrução 1	FI	DI	CO	FO	EI	WO								
Instrução 2		FI	DI	CO	FO	EI	WO							
Instrução 3			FI	DI	CO	FO	EI	WO						
Instrução 4				FI	DI	CO	FO	EI	WO					
Instrução 5					FI	DI	CO	FO	EI	WO				
Instrução 6						FI	DI	CO	FO	EI	WO			
Instrução 7							FI	DI	CO	FO	EI	WO		
Instrução 8								FI	DI	CO	FO	EI	WO	
Instrução 9									FI	DI	CO	FO	EI	WO

PIPELINE

- Se os seis estágios **não** têm duração igual, existe certa espera envolvida em vários estágios
- Uma instrução de desvio condicional pode invalidar diversas buscas de instrução
Igual a ocorrência de interrupção

PIPELINE

	Tempo →							← Penalidade por desvio						
	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instrução 1	FI	DI	CO	FO	EI	WO								
Instrução 2		FI	DI	CO	FO	EI	WO							
Instrução 3			FI	DI	CO	FO	EI	WO						
Instrução 4				FI	DI	CO	FO							
Instrução 5					FI	DI	CO							
Instrução 6						FI	DI							
Instrução 7							FI							
Instrução 15								FI	DI	CO	FO	EI	WO	
Instrução 16									FI	DI	CO	FO	EI	WO

Instrução 3 = um desvio condicional para a instrução 15.

PIPELINE

Tempo
↓

	FI	DI	CO	FO	EI	WO
1	I1					
2	I2	I1				
3	I3	I2	I1			
4	I4	I3	I2	I1		
5	I5	I4	I3	I2	I1	
6	I6	I5	I4	I3	I2	I1
7	I7	I6	I5	I4	I3	I2
8	I8	I7	I6	I5	I4	I3
9	I9	I8	I7	I6	I5	I4
10		I9	I8	I7	I6	I5
11			I9	I8	I7	I6
12				I9	I8	I7
13					I9	I8
14						I9

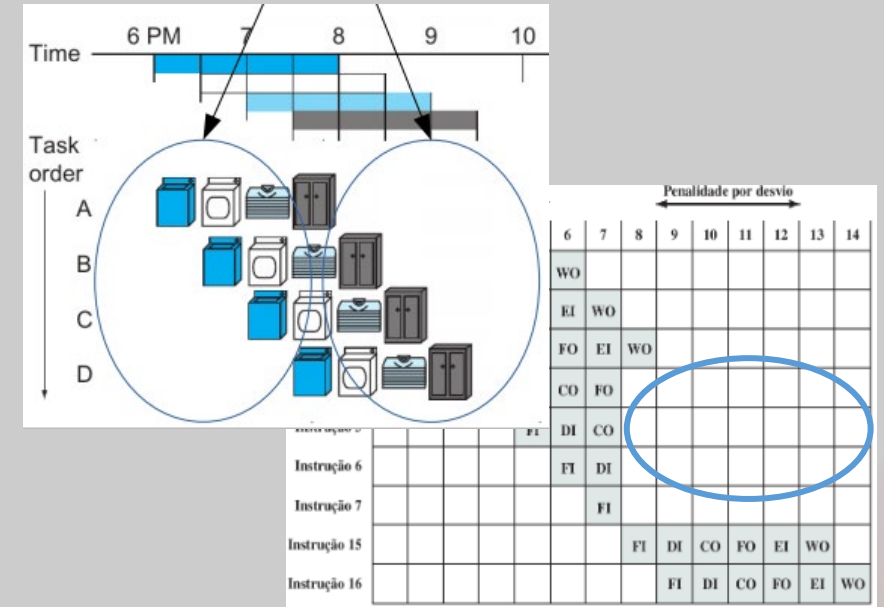
(a) Sem desvios

	FI	DI	CO	FO	EI	WO
1	I1					
2	I2	I1				
3	I3	I2	I1			
4	I4	I3	I2	I1		
5	I5	I4	I3	I2	I1	
6	I6	I5	I4	I3	I2	I1
7	I7	I6	I5	I4	I3	I2
8	I15					I3
9	I16	I15				
10		I16	I15			
11			I16	I15		
12				I16	I15	
13					I16	I15
14						I16

(b) Com desvio condicional

DESEMPENHO

- O tempo para cada instrução é o mesmo que sem pipeline
 - O tempo gasto para uma instrução individual é latência
 - A taxa de inserção de tarefas no pipeline é limitada pela tarefa mais lenta
 - Mas o tempo total de várias instruções é em geral menor
 - ✓ A vazão (*throughput*) pode aumentar
 - ✓ A meta é aumentar a produtividade (de *throughput*)
 - ✓ Pipeline sempre cheio?
- áreas das figuras
- outras situações de prejuízo de desempenho



HAZARDS

- *Hazard* de pipeline ou bolha são situações que podem resultar em um desempenho menor que a ótima
- Quando ocorre? Quando o pipeline, ou parte dele, precisa parar porque as condições não permitem a execução contínua
- Tipos
 - ✓ Recurso (estrutural): 2 ou + instruções que precisam do mesmo recurso
 - ✓ Dados: 2 instruções (leitura ou escrita) querem acessar a mesma posição de operando
 - ✓ Controle (desvio): há necessidade de se tomar uma decisão com base nos resultados de uma instrução enquanto outras estão sendo executadas

Solução: busca antecipada de desvio, uso de *buffer* de laço de repetição (fica no *buffer* n instruções recentemente lidas)

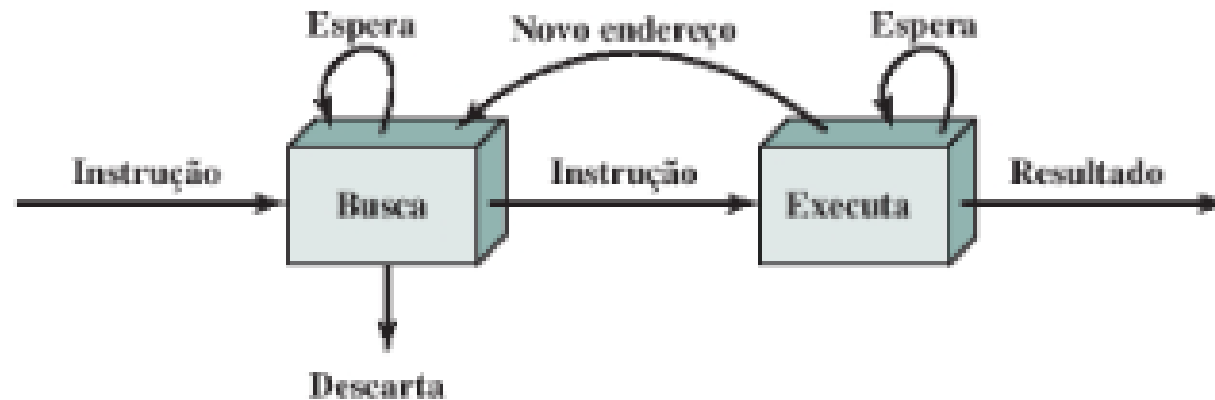
PIPELINE SIMPLES COM ESPERA

Figura 14.9

Pipeline de instruções de dois estágios.



(a) Visão simplificada

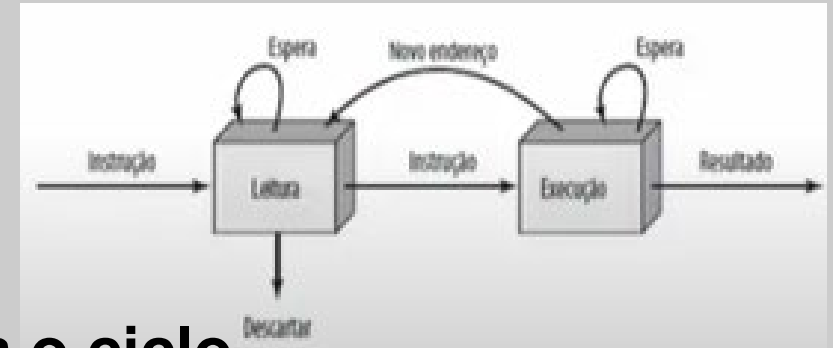


(b) Visão expandida

PIPELINE

- A busca de instruções na memória é um grande gargalo na velocidade de execução do paralelismo da instrução
- Para amenizar esse problema, os computadores têm a capacidade de buscar instruções na memória antecipadamente, de maneira que elas estivessem presentes quando necessárias
- Essas instruções são armazenadas em um conjunto de registradores denominado *buffer* de busca antecipada
- Sempre que necessária, uma instrução pode ser apanhada nesse buffer, em vez de aguardar a conclusão de uma leitura da memória

BUSCA ANTECIPADA



- ***Prefetch (busca avançada):*** A leitura aproveita o ciclo de memória de execução atual para obter a próxima instrução
- ✓ **Requer mais registradores**
- ✓ **Os tempos de leitura não são iguais, pois o tempo de execução é maior (ler e armazenar operandos, instrução de desvio que não conhece o endereço de desvio)**
- ✓ **Tem a regra de que o estágio de leitura sempre obtém a próxima instrução na memória após a instrução de desvio. Se o desvio ocorrer, a instrução é descartada e uma nova é lida**

EM RESUMO

Em função ao número de estágios, o fator de aceleração se aproxima do número de instruções que podem ser introduzidas no pipeline sem desvio.

Quanto maior o número de estágio no pipeline, maior a taxa de execução (aceleração ou *speedup*). Situações que frustram isso:

1- O estágio de pipeline exige esforço extra para movimentar os dados e pode desacelerar o tempo total de execução (custo de implementação e atraso entre estágios)

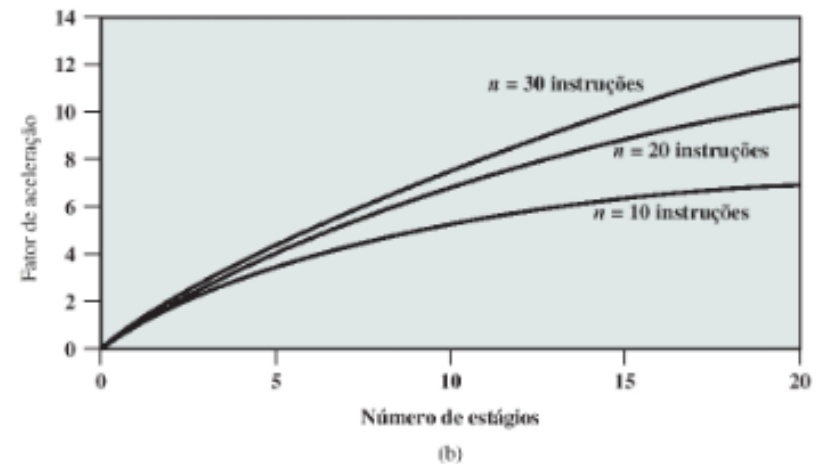
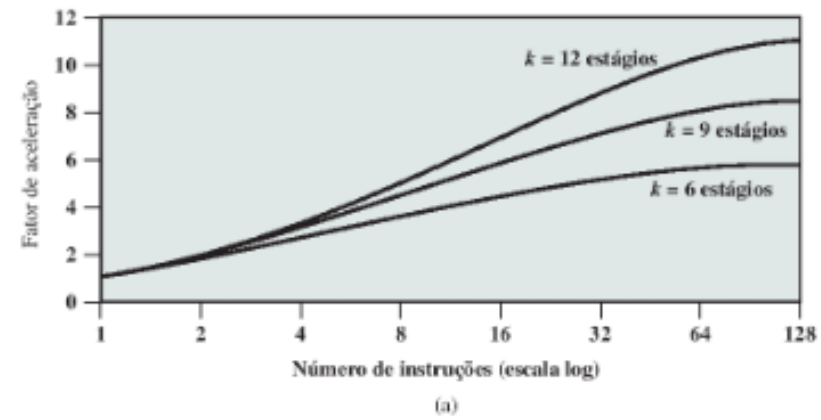
2- A lógica de controle da passagem entre estágios é mais complexa do que os estágios controlados

- Estratégias para evitar *hazards*
- estágios entre 6 e 9 parece ser mais adequado

EM RESUMO

Figura 14.14

Fatores de aceleração com pipeline da instrução.



BIBLIOGRAFIA

- Corrêa, A. G. D (Organizadora). Organização e arquitetura de computadores. Cap. 4. Páginas 169 a 181 (de 187 páginas). Editora Pearson. Edição: 1º (2017). Idioma: Português. ISBN: 9788543020327
- Duncan (1990), "A survey of parallel computer architectures", IEEE Computer, pp. 5-16, Fevereiro, 1990 -Flynn (1972), "Some computer organizations and their effectiveness", IEEE Transactions on Computers, vol. C-21, pp. 948-960, 1972.-
- Stallings, W. Arquitetura e organização de computadores. 427 a 434, 523 a 526 (de 731 páginas). Editora: Editora Pearson. Edição: 10º (2017). Idioma: Português. ISBN: 9788543020532
- Tanenbaum, A & Austin, T. Organização estruturada de computadores. Capítulo 8 - Arquiteturas de computadores paralelos. Páginas 455 a 537 (de 628 páginas). Editora Pearson. Edição: 6º (2013). Idioma: Português. ISBN: 9788581435398"
- Flynn (1972), "Some computer organizations and their effectiveness", IEEE Transactions on Computers, vol. C-21, pp. 948-960, 1972.

SISTEMAS COMPUTACIONAIS

Pipeline